

编译原理 Chapter 1 —概论考点总结

1 课程概述 [p.1-7]

课程地位与重要性 [p.3-4]

- 计算机、软件工程专业的**核心课程** [p.3]
- 编译程序是计算机最重要的**基础软件**之一 [p.3]
- ”软件定义一切”，但离开了编译，程序没有计算机懂 [p.3]
- 国内外将编译原理定义为发展信息技术的**关键课程**之一 [p.4]
- 涵盖：程序语言基础（Programming）、调试工具与技术（Debugging Tools）、编译基础（Compiler Basics）、安全软件开发（Secure Software Development） [p.4]

前导课程 [p.7]

- 组成原理——计算机组成及结构
- 微机原理——汇编语言与机器语言
- 离散数学——推理知识及其完备性
- 数据结构——树、表等的表示与实现
- 操作系统——提供虚拟机和系统调用
- 高级程序设计语言——语言定义和编程

课程要求 [p.5]

了解计算机高级语言源程序被计算机接受、扫描、词法分析、语法分析、语义解释执行的原理与过程；掌握编译的原理和基本算法、各种概念和语言描述。

教材与参考书 [p.2]

- 《编译原理及实践—ARM 架构处理器视角》，陈果等，清华大学出版社（主教材）
- 《编译原理》，陈火旺等，国防工业出版社（经典教材）

2 第一讲：编译器概述 [p.8-25]

什么是编译器 [p.10,17]

- 编译器(Compiler)是一个程序,核心功能是把源代码(Source Code)翻译成目标代码(Target Code) [p.17]
- 源代码: C/C++, Java, C#, HTML, SQL 等高级语言 [p.17]
- 目标代码: x86, IA64, ARM, MIPS 等机器指令 [p.17]
- 翻译流程: 源程序 → 编译器 → 汇编器 → 链接器 → 机器可执行二进制代码 [p.17]

编译器的核心功能 [p.18]

- 静态计算(Static Computation): 编译时, 源程序 → 编译器 → 目标程序
- 动态计算(Dynamic Computation): 运行时, 目标程序 → 计算机 → 计算结果
- 核心要求: 静态计算与动态计算语义相同(Semantics Preserving) [p.18]

编译器 vs 解释器 [p.19]

- 编译器(Compiler): 源程序 → 目标程序, 离线(Offline)执行
- 解释器(Interpreter): 源程序 → 计算结果, 在线(Online)边解释边执行
- 编译器产生目标代码后独立运行; 解释器不产生独立目标代码, 每次执行都需要解释器参与

编程语言发展 [p.11]

- 第 1 代(1GL): 机器语言(0/1 代码)
- 第 2 代(2GL): 汇编语言
- 第 3 代(3GL): 高级语言(C, Java 等)
- 第 4 代(4GL): 工具语言

高级语言嵌入机器语言的实例 [p.12-14]

- 实例 1: 高级语言中嵌入汇编实现读盘/写盘操作 [p.12]
- 实例 2: 高级语言直接使用汇编语句(asm 块), 如修改内存分配 [p.13]
- 实例 3: 高级语言直接使用寄存器(registers), 如调用 MSDOS 中断删除目录 [p.14]

编译器简史 [p.15,20-22]

- 1951-1952 年，**Grace Hopper** 开发了第一个编译器 A-0（后成为美国第一个女将军）[p.20]
- 1954-1957 年，**John Backus** 团队开发了第一个成熟成功的编译器——**Fortran 编译器** [p.21]
- Fortran 编译器的成功在计算机科学中产生巨大影响 [p.22]:
 - 理论上：形式语言、自动机、文法、语法制导的翻译
 - 实践上：算法、数据结构
 - 编译器架构

为什么学习编译原理？ [p.23-24]

- 编译原理集中体现了计算机科学的核心思想：算法、数据结构、软件工程等 [p.23]
- 编译器是其他领域的重要研究基础 [p.23]
- 编译器本身是重要研究领域：新语言设计、大型软件的构造和维护 [p.24]

3 第二讲：编译器结构 [p.26-29]

编译器高层结构 [p.26]

编译器具有非常模块化的高层结构，可看成多个阶段构成的”流水线”：

- 前端 (**Front End**)：词法分析、语法分析
- 后端 (**Back End**)：指令生成、指令优化

编译过程——英译中类比 [p.27-29]

前端各阶段 [p.27-28]：

1. 词法分析 (**Lexical Analysis**)：识别出句子中的单词 → 生成记号流 (Token Stream)
2. 语法分析 (**Syntax Analysis**)：分析句子的语法结构 → 生成语法树 (Syntax Tree)
3. 语义分析 (**Semantic Analysis**)：分析句子的语义 → 生成注释语法树 (Annotated Syntax Tree)

后端各阶段 [p.29]：

1. 中间代码生成 (**Intermediate Code Generation**)
2. 中间代码优化 (**Code Optimization**)
3. 目标代码生成 (**Target Code Generation**)

英译中类比示例 [p.28-29]：

- 原文：”Sheep eat grasses slowly”

- 词法：识别出单词 Sheep, eat, grasses, slowly
- 语法：分析出主语 (Sheep)、谓语 (eat)、宾语 (grasses)、状语 (slowly)
- 语义：sheep 为复数（多只羊），eat 为第三人称复数
- 中间翻译：“羊群缓慢地吃草”
- 优化：幼态版“好多羊羊慢慢地吃好多草草” → 文言版“羊缓食草”

4 第三讲：编译器实例 [p.30-39]

实例一：Sum 语言 → Stack 栈式计算机 [p.31-34]

源语言 Sum [p.32]:

- 两种语法形式：
 - 整型数字： n
 - 加法： $e_1 + e_2$
- 示例：3, 5+6, 7+8+9

目标机器 Stack [p.33]:

- 一个操作数栈
- 两条指令：
 - push n ：往栈顶压入操作数 n
 - add：从栈顶取出两个操作数相加，结果压回栈顶

编译阶段（以 $1+2+3$ 为例）[p.34]:

1. 词法分析：‘1’, ‘+’, ‘2’, ‘+’, ‘3’ → 记号序列
2. 语法分析：验证是否满足语法规则（整型数字 n / 加法 e_1+e_2 ）
3. 语法树构建：构建抽象语法树（AST）
4. 代码生成：生成目标代码 push 1; push 2; add; push 3; add

实例二：C 语言 → ARM64 架构 [p.35-38]

源程序 [p.35]:

```
array[1] = 4 + 2;
```

目标代码 [p.35]:

```
mov w1, #6
str w1, [x0, #4]
```

各阶段详解：

1. 词法分析 (Lexical Analysis) [p.36]：将源代码字符流转换为记号 (Token) 序列：

- `array` $\rightarrow$ `<ID, array>` (标识符)
- `[` $\rightarrow$ `<LBRACKET, [>` (左中括号)
- `1` $\rightarrow$ `<INT_LIT, 1>` (整型字面量)
- `]` $\rightarrow$ `<RBRACKET, ]>` (右中括号)
- `=` $\rightarrow$ `<ASSIGN, =>` (赋值运算符)
- `4` $\rightarrow$ `<INT_LIT, 4>`
- `+` $\rightarrow$ `<PLUS, +>` (加法运算符)
- `2` $\rightarrow$ `<INT_LIT, 2>`
- `;` $\rightarrow$ `<SEMI, ;>` (分号)

2. 语法分析 + 语义分析 [p.37]：构建语法树 (AST)，并进行类型检查等语义分析：

- 根节点：AssignmentExpr (赋值表达式)
- 左子树：ArraySubscriptExpr (数组下标表达式) — Identifier(array)[IntegerLiteral(1)]
- 右子树：AddExpr (加法表达式) — IntegerLiteral(4) + IntegerLiteral(2)
- 语义分析：array 的类型为 array of integer

3. 后端 (中间代码生成 $\rightarrow$ 优化 $\rightarrow$ 目标代码生成) [p.38]：

- 中间代码 (三地址码)：

```
t1 = 4 + 2;
t2 = array + 4;    // 假设int为4字节
*t2 = t1;
```

- 常量折叠优化： $4 + 2 = 6$ ，消除 t1，优化后：

```
t2 = array + 4;
*t2 = 6;
```

- 目标代码 (ARM64)：

```
mov w1, #6
str w1, [x0, #4]
```

更复杂的编译器结构 [p.39]

实际编译器结构更加复杂，包含更多阶段和更多优化。各阶段之间可能有反馈循环。

5 补充：参数传递与存储管理 [p.40-47]

参数传递的三种方式 [p.41-42]

- **传地址 (Call by Reference)**：把实在参数的地址传递给相应的形式参数。形参对地址内容的修改会影响实参 [p.41]
- **传值 (Call by Value)**：把实在参数的值计算出来，放在被调用段可以拿到的地方。形参修改不影响实参 [p.41]
- **传名 (Call by Name)**：过程调用的作用相当于把被调用的过程体抄到调用出现的地方，把其中任一出现的形式参数都替换成相应的实在参数（文字替换、宏替换式语义） [p.42]

经典例题：传名/传值/传地址对比 [p.44-46]

程序：

```
PROGRAM SS(input,output);
VAR A,B: integer;
PROCEDURE P(x,y,z: integer);
  begin y:=y+1; z:=z+x; end;
BEGIN
  A:=2; B:=3;
  P(A+B, A, A);
  writeln('A=', A);
END
```

分析结果 [p.45-46]：

(1) 传名 (Call by Name) [p.45]：将过程体中的形参全部替换为实参表达式（文字替换）：

- $y := y + 1 \rightarrow A := A + 1$ (A 变为 3)
- $z := z + x \rightarrow A := A + (A + B) \rightarrow A := 3 + (3 + 3) = 9$

结果：A=9

(2) 传值 (Call by Value) [p.45]：计算实参的值传给形参：

- $x = 5(A+B), y = 2(A), z = 2(A)$
- 执行过程： $y := 3, z := 7$
- 形参修改不回传实参

结果：A=2

(3) 传地址 (Call by Reference) [p.46]：设变量 $T=A+B=5$ （临时变量）。传递地址：

- x 指向 $T(addr1)$, y 指向 $A(addr2)$, z 指向 $A(addr2)$ (y 和 z 指向同一地址！)
- $y \uparrow := y \uparrow + 1 \rightarrow A := 2 + 1 = 3$
- $z \uparrow := z \uparrow + x \uparrow \rightarrow A := 3 + 5 = 8$ (注意 y 和 z 共享同一变量 A)

结果：A=8

存储管理 [p.43]

- 静态存储分配 (Static Storage Allocation)：编译时就安排好目标程序运行时的全部数据空间，并能确定每个数据项目的单元地址 [p.43]
- 动态存储分配 (Dynamic Storage Allocation)：如果允许递归、可变数据结构，则必须动态分配 [p.43]
 - 栈式 (Stack)：整个程序数据空间安排在一个栈中
 - 堆式 (Heap)：允许自由地申请和退还空间

6 小结 [p.48]

编译器构造与目标相关，目前基本结构：

1. 词法分析 (Lexical Analysis) —— 字符流 → 记号流
2. 语法分析 (Syntax Analysis) —— 记号流 → 语法树
3. 语法树构建 (AST Construction)
4. 代码生成 (Code Generation)

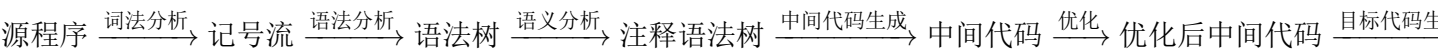
后续会看到更复杂的结构：更多的阶段、更多的优化。

7 核心考点速记

概念对照表

概念	英文术语	要点
编译器	Compiler	源程序 → 目标程序（离线）
解释器	Interpreter	源程序 → 计算结果（在线边解释边执行）
词法分析	Lexical Analysis	字符流 → 记号流（Token）
语法分析	Syntax Analysis	记号流 → 语法树（AST）
语义分析	Semantic Analysis	类型检查、语义验证
中间代码	Intermediate Code	平台无关的中间表示
代码优化	Code Optimization	常量折叠、死代码消除等
目标代码生成	Code Generation	中间代码 → 目标机器代码
传值	Call by Value	传值的副本，修改不影响实参
传地址	Call by Reference	传地址，修改影响实参
传名	Call by Name	文字替换（宏展开式语义）

编译流程速记



关键人物

- **Grace Hopper** (1951-1952): 第一个编译器 A-0, 后成为美国第一位女将军
- **John Backus** (1954-1957): 第一个成熟编译器 Fortran

典型例题考点

- 传名: 形参被实参表达式逐字替换, 注意副作用和多次求值
- 传值: 形参获得实参的副本, 修改不回传
- 传地址: 形参获得实参地址, 注意**别名问题** (多个形参指向同一实参地址)

重要数值

- 编译器前端: 词法分析、语法分析、语义分析
- 编译器后端: 中间代码生成、中间代码优化、目标代码生成
- 参数传递三种方式: 传值、传地址、传名
- 存储分配两种方式: 静态分配、动态分配 (栈式 + 堆式)